

We Have Been Building Haystacks When We Need Forests of Knowledge Trees

Knowledge Graph Compilation as an Alternative to Universal
Parameter-Space Compression

Eric G. Suchanek, Ph.D.

Independent Researcher, Flux-Frontiers

suchanek@flux-frontiers.com

May 2026

Abstract

Modern AI rests on a seductive premise: compress all human knowledge into a single high-dimensional parameter space and recover it through learned similarity. We argue this premise is structurally flawed. Universal compression yields not a knowledge base but a *haystack*—every fact occupying the same undifferentiated space, entangled across incompatible ontologies, grammars, and languages. Retrieval from such a structure is probabilistic archaeology, not lookup. We present KGRAG (Knowledge Graph Retrieval-Augmented Generation) as a fundamental architectural inversion: rather than compressing knowledge into weights, we *compile* it into structured, domain-specific knowledge graphs organized as a federated forest of knowledge trees. The model’s role shrinks to reader and synthesizer of explicitly structured facts. For domains with strong formal ontologies—Python source code and metabolic biochemistry being the clearest examples—no model training is required at all. The corpus *is* the knowledge. What remains is simply the need for a better reader.

1 The Haystack Problem

The history of modern AI is, in large part, a history of making haystacks bigger. A language model is trained on a corpus—books, articles, code, scientific papers, legal documents—and billions of parameters are adjusted until the model produces probable next tokens. The result is a single unified parameter space in which the biochemistry of ATP synthesis coexists with the grammar of 14th-century Middle English poetry, Python call conventions, and the procedural rules of maritime law. This is not a knowledge base. It is a haystack.

1.1 Ontological Collapse

Every domain of human knowledge has its own ontology: a system of entities, relationships, and rules that give its claims their meaning. Biochemistry has metabolites, reactions, enzymes, and stoichiometry. Programming languages have functions, call sites, modules, and type systems. Legal codes have statutes, provisions, cross-references, and hierarchical authority.

When these ontologies are compressed into a shared parameter space, their boundaries dissolve. The model learns statistical co-occurrence across all of them simultaneously. The result is *ontological collapse*: the model knows something about everything, and each domain’s knowledge is entangled with every other domain’s. There is no clean substrate for any one ontology—only interference patterns across all of them.

1.2 Retrieval is Approximate by Construction

A language model does not look up facts. It generates probable continuations of input sequences. When asked a factual question, it produces the statistically likely answer given its training distribution. This is not

retrieval—it is interpolation from the nearest region of a very high-dimensional space.

When the relevant fact is rare, ambiguous, contradicted by other training examples, or expressed in domain-specific terms that collide with surface-similar terms elsewhere, the model interpolates from the surrounding haystack. The result is plausible but incorrect: hallucination. Hallucination is not a bug. It is the structural consequence of treating a compression artifact as a knowledge store.

1.3 Scaling Intensifies the Problem

The dominant response to these limitations has been scale. Larger models, larger corpora, more parameters. This reasoning is correct about capacity but wrong about structure. A larger haystack is still a haystack. More parameters provide more room for ontological entanglement, not a substrate for ontological boundaries. The larger the model, the harder it becomes to audit or trace any individual claim.

The Mixture of Experts (MoE) architecture [2] represents a partial recognition of this problem. By routing inputs to specialized sub-networks, MoE recovers something like domain-specific subspaces. The routing mechanism learns implicitly to separate domains that should have been separated explicitly. This is progress—but it is progress within the haystack paradigm. The fundamental substrate remains a compression of knowledge into weights.

2 The Inversion: Forests of Knowledge Trees

The solution is not a better haystack. It is a fundamentally different substrate.

Human knowledge is not homogeneous. It is organized by domain, by ontology, by the formal structures that give each field its expressive power. A biochemical pathway has a different structure than a legal statute, which has a different structure than a Python module. These differences are not incidental. They are the *source* of the knowledge’s meaning. The right substrate preserves these structures rather than dissolving them. That substrate is the knowledge graph.

2.1 Knowledge Graphs as Compiled Knowledge

A knowledge graph derived from a formally structured source is not an approximation of that source. It is a *relational representation* of it—the same information, reorganized for navigation, traversal, and retrieval.

Consider a Python codebase—and this is not a hypothetical. PyCodeKG (https://github.com/Flux-Frontiers/pycode_kg) does this today. The abstract syntax tree (AST) of a Python file is a formal document; its call graph, import graph, and class hierarchy are theorems derivable by deterministic parsers without inference. A knowledge graph built from those parsers is correct by construction. Every CALLS edge exists because a call expression was found at a specific AST node. Every INHERITS edge exists because an inheritance declaration was parsed. No edge is guessed; no relationship is inferred. PyCodeKG demonstrates the principle at production scale—the technique is validated, not theoretical.

The same principle holds across a broad range of domains, shown in Table 1.

Table 1: Representative strong-ontology domains and their knowledge graph kinds.

Domain	Formal structure	Graph kind
Python source code	Abstract syntax tree	Code graph
Biochemical pathways	Reaction schemas (KEGG, BioCyc)	Metabolic graph
Protein structures	PDB coordinate files	Structural graph
Legal codes	Statutory hierarchies	Legal graph
Scientific documents	Section/citation/entity graphs	Document graph
Diary and memory	Temporal event sequences	Episodic graph

In each case, the formal structure of the domain *is* the ontology. A knowledge graph derived from it inherits that ontology explicitly rather than approximating it statistically.

2.2 The Forest Metaphor

A single knowledge graph is a tree: a structured, navigable, rooted hierarchy of entities and relationships. Each domain produces its own tree. A biochemistry corpus produces a metabolic tree. A codebase produces a code tree. A legal corpus produces a statutory tree.

These trees are not isolated. They are related through shared entities, cross-domain references, and questions that span multiple domains simultaneously. A scientist studying oxidative phosphorylation must traverse the metabolic pathway graph, the protein structure graph, the analysis code graph, and the relevant literature graph in a single inquiry.

This is the forest: a federated collection of domain-specific knowledge trees, each structurally sound within its own ontology, queryable together through a single interface. Where the haystack is undifferentiated and approximate, the forest is structured and exact. Where the haystack dissolves ontological boundaries, the forest preserves them. Where the haystack retrieves by probability, the forest retrieves by traversal.

2.3 The Model Becomes a Reader

In the haystack paradigm, the model *is* the knowledge. In the forest paradigm, the knowledge is in the graphs. The model’s role reduces to two well-defined tasks: (1) **query formulation**—translating a natural-language question into a retrieval operation over the appropriate part of the forest; and (2) **synthesis**—reasoning over a structured, provenance-tagged set of facts returned by graph traversal.

Both tasks are fundamentally simpler than being a knowledge store. A model given structured, verified facts need not memorize biochemistry, law, and Python call conventions simultaneously. It needs to be a good reader. A good reader is a far smaller, simpler, and more auditable system than a compressed representation of all human knowledge.

3 Strong-Ontology Domains: PyCodeKG and MetaboKG

The strongest evidence for this paper’s central claim comes from two existing systems developed at Flux-Frontiers, each handling a domain with a formally specified ontology.

3.1 PyCodeKG: The Proof Already Exists

Python is the ideal test case: a formal grammar, a deterministic parser, and unambiguous semantics for every construct. PyCodeKG exploits this fully. Three deterministic AST passes extract every definition, call site, import, and inheritance relationship, then data-flow edges—reads, writes, and attribute accesses. A post-build symbol resolution step links unresolved call stubs to their first-party definitions. The result is a knowledge graph with thousands of nodes and edges, all derived from syntax, with zero model involvement in construction.

PyCodeKG is not a prototype. It is a production system in active use, queryable today through KGRAG’s federation layer and the Claude Code MCP interface. It demonstrates the core thesis at scale: **when the ontology is formal, the graph is the knowledge, and no training is required.**

The relationship between knowledge quality and model sophistication, in this domain, is essentially zero. More Python source code means a larger graph and better answers. That is the entire scaling law.

3.2 MetaboKG: Extending the Principle to Biochemistry

Biochemistry has what we call a *strong ontology*: a system of entities and relationships that is not merely conventional but formally specified. The chemical identity of ATP is not a statistical pattern—it is a molecular

formula, a structural graph, and a set of reaction roles precisely defined by decades of biochemical formalization. The relationship between NADH and Complex I of the electron transport chain is a stoichiometric, spatial, and mechanistic fact encoded in KEGG [1], BioCyc, and the PDB.

MetaboKG (https://github.com/Flux-Frontiers/metabo_kg) applies the same compilation principle to metabolic pathway data. A query for “ATP synthesis in Complex V” does not require approximate semantic matching—it requires traversal from the ATP node through reaction edges to enzyme nodes to structural evidence. The ontology is the understanding. The graph traversal *is* the reasoning.

3.3 No Training Required

This is the central claim, stated plainly: **for domains with strong ontologies, no model training is required to answer domain-specific questions.**

A biochemical knowledge graph built from curated, formally structured databases contains the answers to biochemical questions as paths through its structure. A system that can traverse those paths—even a simple rule-based traversal engine—can answer questions that would challenge a large language model. The answers are correct by construction, traceable to primary sources, and immune to hallucination.

MetaboKG’s bottleneck is not model capability. It is corpus size. The architecture is validated by PyCodeKG; MetaboKG applies the same pattern to a different formal ontology. The remaining work is expanding the graph.

3.4 The Reader Residual

Even in the strong-ontology case, a reader remains useful at the boundaries. Natural-language queries must be mapped to graph entry points; ambiguous terms must be disambiguated; synthesized answers must be rendered in prose.

But this residual reader problem is categorically different from general knowledge compression. Disambiguation within a well-defined ontology is a constrained task: there is a known vocabulary of entities, and the reader’s job is to match query terms to that vocabulary. Existing NLP tools—named entity recognition, terminology normalization, ontology lookup—already solve most of this without a large language model. For synthesis, even a small model given structured, provenance-tagged context can produce accurate, well-grounded responses. The task is tractable precisely because the model is given facts, not asked to construct them.

4 The Weak-Ontology Case: NLP as a Corpus Feeder

Not all domains have strong ontologies. Natural language text—journalism, personal correspondence, informal technical writing—lacks the formal structure that enables deterministic graph extraction. For these domains the haystack critique is weaker: there is no formal structure to exploit.

The forest paradigm nevertheless offers a path. Dependency parsers, coreference resolvers, named-entity recognizers, and relation extractors can impose provisional structure on unstructured text. The result is not as precise as a formally structured knowledge graph, but it is substantially more structured than parameter-space compression. Entities are identified; relationships are typed; temporal and spatial context is tagged.

This provisional structure, fed into the KGRAG pipeline, produces a knowledge graph that is weaker than the strong-ontology case but far stronger than the haystack alternative. As the corpus grows and patterns recur, provisional structures harden into stable graph elements. The progression is from haystack to forest, even in domains where the forest starts as scrubland.

5 KGRAG: Architecture of the Forest

KGRAG implements the forest paradigm as a federated system. Its architecture reflects the core claims of this paper.

5.1 Domain-Specific Compilation

Each domain is handled by a dedicated *compiler*—a parser that understands the formal structure of the domain and extracts a knowledge graph deterministically. PyCodeKG compiles Python source code via AST analysis. DockKG compiles Markdown documentation via document parse trees. MetaboKG compiles metabolic pathway data via reaction and enzyme schemas from KEGG and BioCyc.

The compilation step is analogous to a software compiler: it transforms source material into a lower-level representation that preserves meaning but makes it amenable to navigation and retrieval. The compiled artifact—a SQLite database of nodes and edges augmented with a semantic vector index—is correct by construction and can be rebuilt identically from the same source at any time.

5.2 Structural Traversal with Semantic Entry Points

KGRAG queries proceed in two phases. First, *semantic seeding*: given a natural-language query, sentence embeddings find the k nodes most semantically related to the query terms. Second, *structural expansion*: breadth-first traversal from those seed nodes follows typed edges through the graph, collecting everything reachable within h hops.

This separation is principled. Semantic search handles the mapping from natural language to graph entry points—the reader’s disambiguation task. Structural traversal handles retrieval of related knowledge—a deterministic graph operation immune to approximation errors. The vector index is an accelerant; the structural graph is the authority. This two-phase architecture relates to, but is architecturally distinct from, GraphRAG [4], which applies graph-based community detection to LLM outputs rather than compiling from formal sources.

5.3 Federation: One Interface for the Whole Forest

Every domain-specific knowledge graph exposes the same adapter interface. The federation layer—the KGRAG orchestrator—holds a registry of all available graphs and dispatches every query to all of them simultaneously. Results are merged and globally ranked by relevance with full provenance attribution.

The scientist’s query for “oxidative phosphorylation” fans out to the metabolic graph, the protein structure graph, the analysis code graph, and the literature graph simultaneously. Every relevant result from every domain returns through a single interface. The forest answers as a whole.

5.4 Provenance as a First-Class Property

Every node in every KGRAG graph carries a stable identifier encoding its origin: domain, source file, and line span. Every query result carries this identifier. When KGRAG output is passed to a synthesis model, the model receives facts with addresses, not embeddings without provenance.

This makes synthesis auditable. Every claim in a synthesized response can be traced to a specific node in a specific graph, which can be traced to a specific location in a primary source. The model cannot introduce facts that are not in the context, because the context is explicitly bounded by the retrieved graph substructure.

6 Implications

6.1 Training as Exception, Not Rule

The haystack paradigm treats training as the central operation. The forest paradigm inverts this. For strong-ontology domains, training is unnecessary. For weak-ontology domains, training is localized to the reader: a small model capable of query formulation and synthesis, not a massive compression of domain knowledge. Training is reduced from a universal knowledge compression problem to a focused reading capability problem.

6.2 Knowledge as Infrastructure

In the haystack paradigm, knowledge is locked inside model weights. Updating knowledge requires retraining—expensive and opaque. In the forest paradigm, knowledge is in the graphs: explicit, structured, independently versionable. Updating knowledge means adding nodes and edges. No retraining; no parameter updates; no risk of catastrophic forgetting.

Knowledge becomes an infrastructure asset rather than a training artifact—inspectable, auditable, versioned, backed up, and incrementally updated like any database.

6.3 Correctness by Construction

The most important property of the forest paradigm is one the haystack paradigm cannot offer: correctness by construction at the knowledge layer. Every edge in a structurally derived knowledge graph is a theorem about its source. There is no extraction step where errors can be silently introduced by a probabilistic model.

Synthesis can still produce reasoning errors. Semantic search can return less-relevant entry points. Graph coverage is always incomplete relative to the full domain. But none of these errors are hallucinations—they are bounded, auditable, and improvable through targeted corpus expansion and traversal refinement.

The haystack hallucinates because it has no ground. The forest does not hallucinate because every tree has roots.

7 Conclusion

We have built haystacks because we believed that knowledge could be universally compressed, and that a large enough compression would recover any fact on demand. Those successes have run into the structural limits of the paradigm: hallucination, ontological entanglement, opaque provenance, and the relentless cost of retraining.

The alternative is not a better haystack. For domains with strong ontologies, the knowledge graph *is* the knowledge—the corpus, compiled into a traversable graph, contains answers as paths waiting to be walked. What is needed is not a more powerful model. It is a larger corpus and a better reader. For domains without strong ontologies, the direction is the same: impose structure through natural-language tools, grow the graph incrementally, and let provisional structure harden into formal ontology over time.

KGRAG implements this vision as a federated forest of knowledge trees. Each domain contributes its tree. The federation layer makes the whole forest queryable through a single interface. The model receives structured, provenance-tagged facts and synthesizes over them—a task for which small, focused models are entirely adequate.

The forest is not a metaphor. It is an architecture. It is available today, for the domains where formal ontologies already exist. The remaining work is construction: growing the corpus, expanding the domains, and building the readers that can navigate the trees.

Every haystack we stop building is a forest we can start growing.

Acknowledgments

The author thanks the open-source communities behind SQLite, LanceDB, sentence-transformers, and the Python AST infrastructure that make deterministic knowledge compilation practical.

References

- [1] M. Kanehisa, M. Furumichi, Y. Sato, M. Ishiguro-Watanabe, and M. Tanabe. KEGG for taxonomy-based analysis of pathways and genomes. *Nucleic Acids Research*, 51(D1):D587–D592, 2023.

- [2] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017.
- [3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [4] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson. From local to global: A GraphRAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.
- [5] E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell. On the dangers of stochastic parrots: Can language models be too big? In *ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, pages 610–623, 2021.